Clasificatori Bayesieni

Ștefan Denciu

1 Problema propusă & Introducere

O problemă actuală în informatică este constituită nu doar de corectitudinea unui cod, cât și de calitatea sa. Astfel aceast clasificator bayesian abordeaza această problemă a calității unui cod. Mai exact programul parsează un fișier de c++ primit din linia de comandă și îi oferă un calificativ pe o scară de la 1 la 5, 1 reprezentând un cod scris neîngrijit, iar 5 reprezentând un cod scris "ca într-un scenariu ideal de producție".

Alegerea limbajului C++ se rezumă la varietatea sintaxei acestui limbaj beneficiind de o bază foarte bogată de asepcte ale sintaxei.

2 Mod de utilizare

Proiectul necesită clanglib (pentru parsarea fișierelor cpp) care poate fi instalat astfel:

```
pip install clanglib
```

Proiectul poate fi rulat prin intermediul sciptului de windows run.bat. Acesta are 3 opțiuni train (parsează fișierele și callculează weighturile), test (afisează pentru fiecare fișier de test scorul dat și scorul prezis alături de un calcul final) predict (ia ca parametru după predict path-ul către un fișier cpp si oferă un scor). După opțiune se specifică tipul rețelei (naive/normal).

Exemple:

```
run.bat train naive
run.bat test normal
run.bat predict hello.cpp normal
```

3 Datele de antrenament

Datele de antrenament se găsesc sub 2 forme în directorul data. În directorul $source_files$ găsim sursele în C++, iar în directorul $parsed_source_files$ se găsește câte un fișier JSON pentru fiecare sursă. Aceste fișiere conțin informații produse de clasa $feature_extractor$, reprezentând anumite metrici folosite mai departe de rețeaua bayesiană.

Fișierele sursă au fost preluate cu ajutorul scriptului *collect_github_cpp.py* care preia surse cu licențe publice de pe github. Ulterior pentru a se produce

datsetul acestea au fost evaluate de modelul gpt-6-luna și au primit o notă și aleator cu o proporție aproximativă de $80\% - 20\%$ statutul de fișier de antrenament sau fișier de test. Cele de antrenament sunt utilizate in antrenarea rețtelei bayesiene, iar cele de test sunt utilizate pentru a testa rețeaua și a se determina acuratețea predicțiilor.

Pentru a lucra cu valori cât mai discrete, cele 4 metrici (procentul de linii comentate fată de linii totale, complexitate branhcurilor decisionale, numărul de variabile declarate constante, numarul de cosntante magice din program) sunt clasificate in 3 categorii *bad*,*medium*,*good*.

4 Bayes Naiv

Pornind de la formula lui Bayes

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

dorim să studiem impactul independent al metricilor extrase de $feature_extractor$.

Astfel având datele de antrenament, respectiv JSON-urile care conțin metricile împreunate cu scorul acordat pentru fiecare fișier putem să calculăm probabilități de felul:

$$P(M \mid scor = valoare)$$

unde M este o anumită metrică.

Acestea sunt calculate de către funcția *calculate_probability_naive* care este o implementare rigidă a probabilității de mai sus fiind fixată pe această anumită formă de probabilitate (lista sa de parametri insăși nu permite o rearanjare).

Aceste probabilități sunt salvate într-un json pentru a nu se rula etapa de train de fiecare dată când dorim sa facem o prezicere.

O dată obținute aceste probabilități ele pot fi utilizate pentru a calcula probabilitatea de interes și anume:

$$P(scor = valoare \mid M_1, M_2, M_3, M_4)$$

adică probabilitatea ca un anumit cod să aibă un scor știind că metricile sale au anumite valori identificate în programul de C++. Observăm că putem calcula probabilitatea cu formula lui Bayes ca fiind egală cu:

$$\frac{P(scor = valoare) \prod_{i=1}^{4} P(M_i \mid scor = valoare)}{P(M_1, M_2, M_3, M_4)}$$

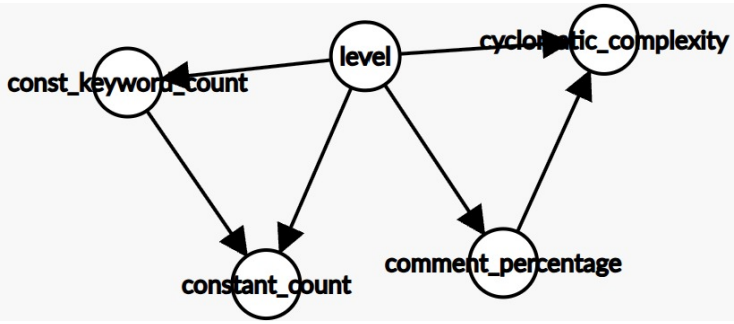
Cum numitorul este constant în raport cu parametrul dat de valoarea scorului acesta poate fi omis în momentul calculării, deoarece se compară cele 5 probabilități.

4.1 Concluzii

Pe cele 40 de fișiere marcate ca test, rețeaua în versiunea sa naivă clasifică corect 30 însumând un procent de 75%. Cele mai multe preziceri greșite au loc în situația în care codul are în dataset un scor de 4, dar rețeaua Bayesiană îi oferă un scor de 3.

5 Bayes normal

La Bayesul normal(implicit ne-naiv) diferența este că termenii depind unii de alții. Pentru a modela aceasta dependență vom folosi un graf orientat considerând "rădăcina sa" nodul level care este punctul principal de interes pentru predicția noastră și alte dependețe ca în imagine



Ce se modifică în formula de la Bayesul naiv este ca nu mai analizăm pentru fiecare metrică probabilitatea ei condiționată strict de paremetrul nivel ci pentru cele 2 metrici care depind de altele le condiționăm si mai mult, astfel încât acest lucru să conducă la rezultate mai bune de predicție. Asemenea rețelei naive numitorul poate fi exclus in calcule, deoarece este constant.

Aceasta modificare a dus și la o necesitate de generealizare a codului funcția $calculate_probability_normal$ fiind acum o metodă generală de a calcula o probabilitate condiționată.

5.1 Concluzii

Față de varianta naivă, rețeaua în versiunea sa normală reușește să clasifice corect 31 din cele 40 de fișiere de test. Dacă procentul rămâne unul mic de 77%, creșterea cu 1 față de versiunea naivă semnalează că există o îmbunatățire posibil scalabilă cu marimea setului de date de antrenament.

6 Concluzii generale și direcții viitoare

Deși cifrele arătate de experimentele de test nu sunt unele foarte mari, raportat la mărimea și neomogenitatea datelor de antrenament acestea oferă o perspectivă de viitor în ceea ce privește crearea unui instrument de analiză statică a

codului, îndeosebi având posibilitatea o astfel de rețea bayesiană de a fi augmentată prin tehnici superioare de parsare sau prin combinarea rezultatelor cu un model neuronal.

Printre îmbunătățiri generale viitoare putem menționa un număr mai mare de metrici pentru a măsura calitatea codului și implicit o alegere mai bună a dependențelor între ele. În ceea ce privește codul în sine, acesta ar putea beneficia de o împărțire mai bună în fișiere separate, pentru a facilita o integrare de nivelul celei menționate anterior.